

```

import torch
from datasets import load_dataset
from transformers import (GPT2LMHeadModel, GPT2Tokenizer, Trainer,
                          TrainingArguments, DataCollatorForLanguageModeling)
import matplotlib.pyplot as plt
# Use tiny sample dataset (only 200 examples as per your requirement - no huge dataset)
print("Using device: cuda")
dataset = load_dataset("amazon_polarity", split="train[:200]")
print(f"Dataset loaded with {len(dataset)} examples")
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
model = GPT2LMHeadModel.from_pretrained("gpt2")
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = tokenizer.eos_token_id
def tokenize_function(examples):
    texts = [t + " " + c for t, c in zip(examples['title'], examples['content'])]
    return tokenizer(texts, truncation=True, max_length=256, padding="max_length")
tokenized = dataset.map(tokenize_function, batched=True,
                        remove_columns=dataset.column_names).train_test_split(0.1)
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
print("Tokenization completed.")
print("Starting fine-tuning of GPT-2 on product reviews...")
training_args = TrainingArguments(
    output_dir="./gpt2-finetuned", num_train_epochs=3,
    per_device_train_batch_size=8, eval_strategy="epoch",
    learning_rate=5e-5, weight_decay=0.01, report_to="none")
trainer = Trainer(model=model, args=training_args,
                  train_dataset=tokenized["train"], eval_dataset=tokenized["test"],
                  data_collator=data_collator)
trainer.train()
trainer.save_model("./gpt2-finetuned")
print("Fine-tuning completed and model saved!")
# === SHOW GENERATED PRODUCT REVIEWS (as per lab manual) ===
print("\n=== Generated Product Reviews ===")
prompt = "This wireless earbuds are"
inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
with torch.no_grad():
    outputs = model.generate(*inputs, max_new_tokens=80,
                             temperature=0.8, top_p=0.9, do_sample=True,
                             repetition_penalty=1.2)
generated = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(f"Prompt: This wireless earbuds are")
print(f"Generated: {generated}")
print("Fine-tuning completed and model saved!")

```

```
Using device: cuda
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md:      6.81k/? [00:00<00:00, 559kB/s]

amazon_polarity/train-00000-of-00004.par(...): 100%                                260M/260M [00:05<00:00, 64.6MB/s]

amazon_polarity/train-00001-of-00004.par(...): 100%                                258M/258M [00:05<00:00, 78.7MB/s]

amazon_polarity/train-00002-of-00004.par(...): 100%                                255M/255M [00:08<00:00, 635MB/s]

amazon_polarity/train-00003-of-00004.par(...): 100%                                254M/254M [00:04<00:00, 112MB/s]

amazon_polarity/test-00000-of-00001.parq(...): 100%                                117M/117M [00:02<00:00, 86.2MB/s]

Generating train split: 100%                                3600000/3600000 [00:16<00:00, 162035.26 examples/s]

Generating test split: 100%                                400000/400000 [00:00<00:00, 432570.24 examples/s]

Dataset loaded with 200 examples

tokenizer_config.json: 100%                                26.0/26.0 [00:00<00:00, 2.40kB/s]

vocab.json:      1.04M/? [00:00<00:00, 8.41MB/s]

merges.txt:      456k/? [00:00<00:00, 3.00MB/s]

tokenizer.json:   1.36M/? [00:00<00:00, 13.3MB/s]

config.json: 100%                                665/665 [00:00<00:00, 73.8kB/s]

model.safetensors: 100%                                548M/548M [00:06<00:00, 61.5MB/s]

Loading weights: 100%                                148/148 [00:00<00:00, 498.95it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2
Key | Status | |
-----+-----+---
h.{0...11}.attn.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

generation_config.json: 100%                                124/124 [00:00<00:00, 8.01kB/s]
```